



TẬP ĐOÀN BƯU CHÍNH VIỄN THÔNG VIỆT NAM
VNPT IT - TRUNG TÂM DMS
Bộ phận Phát triển Giải pháp AI

BÁO CÁO NGHIÊN CỨU VÀ PHƯƠNG ÁN TRIỂN KHAI HỆ THỐNG TEXT-TO-SQL DỰA TRÊN KỸ THUẬT GOOGLE CLOUD VÀ NỀN TẢNG VANNA

Định nghĩa, Phương pháp luận, Kiến trúc tham chiếu
và Kế hoạch Khảo nghiệm Hệ thống Hỏi đáp Dữ liệu

PHẠM VI NGHIÊN CỨU VÀ KHẢO NGHIỆM

Báo cáo tổng hợp hai nguồn kỹ thuật chính: bài viết *Getting AI to write good SQL: Text-to-SQL techniques explained* của Google Cloud và mã nguồn mở Vanna. Mục tiêu là xây dựng phương pháp chuyển câu hỏi nghiệp vụ thành SQL có kiểm soát, có thể giải thích, kiểm thử và triển khai trên hạ tầng nội bộ.

Tóm tắt thông số nghiên cứu	
Phương pháp luận:	Context engineering, truy hồi ngữ nghĩa, làm rõ ý định, kiểm chứng SQL và tự hiệu chỉnh
Nền tảng tham chiếu:	Vanna 2.0, FastAPI, Ollama, PostgreSQL trên Neon, công cụ thực thi SQL và giao diện hội thoại
Đầu ra hệ thống:	SQL, bảng dữ liệu, biểu đồ và tóm tắt ngôn ngữ tự nhiên
Nguyên tắc an toàn:	Chỉ đọc, phân quyền theo người dùng, kiểm tra cú pháp, giới hạn tài nguyên và nhật ký kiểm toán

Người thực hiện:
Lê Đức Anh
Trung tâm DMS

Mục lục

1	Tóm tắt kết quả cho Lãnh đạo (Executive Summary)	2
2	Cơ sở lý thuyết và các định nghĩa	3
3	Những thách thức cốt lõi của Text-to-SQL	4
4	Phương pháp luận nâng cao độ chính xác	5
5	Kiến trúc Vanna và ánh xạ với phương pháp Google Cloud	7
6	Phương án triển khai hệ thống đề xuất	8
7	Phương pháp đánh giá và tiêu chí nghiệm thu	10
8	Kế hoạch thực hiện và quản trị rủi ro	12
9	Kết luận và đề xuất hành động	12
	Tài liệu tham khảo	14

1. Tóm tắt kết quả cho Lãnh đạo (Executive Summary)

Text-to-SQL là năng lực chuyển câu hỏi ngôn ngữ tự nhiên của người dùng thành câu lệnh SQL để truy vấn dữ liệu. Giá trị chính của công nghệ là rút ngắn thời gian từ câu hỏi nghiệp vụ đến thông tin ra quyết định, đồng thời giảm phụ thuộc vào nhóm chuyên gia SQL. Tuy nhiên, một mô hình ngôn ngữ đơn lẻ không tự biết cấu trúc dữ liệu, định nghĩa nghiệp vụ và quy tắc phân quyền của từng đơn vị. Vì vậy, chất lượng hệ thống phụ thuộc chủ yếu vào **ngữ cảnh được truy hỏi, khả năng làm rõ ý định và cơ chế kiểm chứng SQL**, không chỉ phụ thuộc vào kích thước LLM [1, 4].

Bài viết của Google Cloud phân loại ba nhóm khó khăn cốt lõi: cung cấp ngữ cảnh đặc thù doanh nghiệp; hiểu đúng ý định của người dùng; và khắc phục giới hạn sinh mã của LLM, đặc biệt là khác biệt phương ngữ SQL. Các kỹ thuật đề xuất gồm truy hỏi và xếp hạng bảng/cột, học trong ngữ cảnh bằng ví dụ, liên kết dữ liệu, lớp ngữ nghĩa, phân giải thực thể, câu hỏi làm rõ, dry-run, viết lại truy vấn và self-consistency [1].

Vanna là nền tảng Python mã nguồn mở cho chuỗi xử lý *câu hỏi tự nhiên* $\rightarrow$ *SQL* $\rightarrow$ *kết quả*. Ở Vanna 2.0, hệ thống được tổ chức theo kiến trúc agent và công cụ, có giao diện web streaming, bảng và biểu đồ, nhận diện người dùng, kiểm soát quyền và tích hợp nhiều LLM/cơ sở dữ liệu [2]. Vanna cung cấp khung triển khai, trong khi phương pháp của Google Cloud cung cấp các nguyên tắc nâng độ chính xác. Hai nguồn bổ sung cho nhau.

Khảo nghiệm trong báo cáo đã được hiện thực bằng FastAPI, Vanna 2.0, Ollama với model llama3.2 và PostgreSQL được quản lý trên Neon. Cơ sở dữ liệu mẫu gồm 10 bảng quan hệ, 10.000 đơn hàng, 20.000 dòng sản phẩm và view ngữ nghĩa `sales_enriched`. Kết nối ứng dụng sử dụng `PostgresRunner` và biến môi trường `DATABASE_URL`; kết quả kiểm thử xác nhận agent truy vấn trực tiếp dữ liệu trên Neon thay cho SQLite.

Đề xuất cốt lõi

Triển khai theo kiến trúc hai vòng bảo vệ: (1) xây dựng ngữ cảnh đúng bằng metadata, tài liệu nghiệp vụ và ví dụ SQL đã xác nhận; (2) kiểm chứng truy vấn bằng parser, chính sách chỉ đọc, dry-run/thực thi giới hạn và vòng sửa lỗi. Với câu hỏi mơ hồ, hệ thống phải hỏi lại thay vì tự suy đoán.

Bảng 1: Kết luận định hướng từ hai nguồn nghiên cứu

Khía cạnh	Google Cloud	Vanna
Trọng tâm	Kỹ thuật nâng độ chính xác text-to-SQL	Khung ứng dụng và vận hành agent dữ liệu
Ngữ cảnh	Truy hỏi bảng/cột, ví dụ, lớp ngữ nghĩa, lịch sử	DDL, tài liệu, cặp câu hỏi-SQL, bộ nhớ và system prompt
Kiểm chứng	Parser, dry-run, reprompting, self-consistency	Công cụ SQL, middleware, lifecycle hook, audit và UI có cấu trúc
Đầu ra	Truy vấn đúng ngữ nghĩa và đúng phương ngữ	SQL, bảng, biểu đồ, tóm tắt streaming

2. Cơ sở lý thuyết và các định nghĩa

2.1. Định nghĩa Text-to-SQL

Text-to-SQL là bài toán ánh xạ một phát biểu ngôn ngữ tự nhiên q và ngữ cảnh dữ liệu C thành câu lệnh SQL s :

$$f_{\theta}(q, C, d, p) \longrightarrow s$$

trong đó d là phương ngữ SQL (SQLite, PostgreSQL, BigQuery, MySQL,...), p là chính sách truy cập, và θ biểu diễn mô hình sinh. Một hệ thống hoàn chỉnh không dừng ở việc sinh chuỗi SQL; nó phải xác nhận SQL chạy được, tuân thủ quyền, trả đúng tập kết quả và diễn giải kết quả một cách trung thực.

2.2. Ngữ cảnh nghiệp vụ (Business-specific Context)

Ngữ cảnh là toàn bộ thông tin ngoài câu hỏi giúp mô hình hiểu đúng dữ liệu:

- **Ngữ cảnh cấu trúc:** schema, DDL, khóa chính/khóa ngoại, kiểu dữ liệu, quan hệ giữa bảng và phương ngữ SQL.
- **Ngữ cảnh ngữ nghĩa:** ý nghĩa cột, định nghĩa KPI, quy tắc lọc, cách hiểu các mã nội bộ và quy ước thời gian.
- **Ngữ cảnh bằng ví dụ:** cặp câu hỏi-SQL đã được chuyên gia xác nhận, truy vấn phổ biến và mẫu join chuẩn.
- **Ngữ cảnh dữ liệu:** giá trị mẫu, phân bố, từ đồng nghĩa và ánh xạ thực thể từ ngôn ngữ người dùng sang giá trị trong cơ sở dữ liệu.

Google Cloud nhấn mạnh rằng fine-tuning cho từng schema thường không mở rộng tốt vì schema và dữ liệu thay đổi liên tục; thay vào đó, nên truy hồi ngữ cảnh cần thiết tại thời điểm hỏi [1].

2.3. RAG và truy hồi có định hướng SQL

Retrieval-Augmented Generation (RAG) là cách truy hồi một tập ngữ cảnh liên quan trước khi gọi LLM. Với text-to-SQL, kho truy hồi thường chứa:

1. DDL và mô tả bảng/cột;
2. tài liệu định nghĩa nghiệp vụ;
3. cặp câu hỏi-SQL đúng;
4. lịch sử truy vấn và phản hồi của người dùng.

Vanna 1.x gọi quá trình nạp các thành phần trên là “training”, nhưng đây không nhất thiết là huấn luyện lại trọng số LLM. Về bản chất, hệ thống lưu metadata/embedding vào lớp truy hồi và đưa các mục liên quan vào prompt khi có câu hỏi mới [3, 4].

2.4. Lớp ngữ nghĩa (Semantic Layer)

Lớp ngữ nghĩa là cầu nối giữa cấu trúc dữ liệu kỹ thuật và ngôn ngữ nghiệp vụ. Ví dụ, “doanh thu thuần” có thể được chuẩn hóa thành:

$$\text{Net Revenue} = \sum (\text{quantity} \times \text{unit price} \times (1 - \text{discount}))$$

với điều kiện loại đơn hủy/hoàn. Khi định nghĩa này được quản trị tập trung, mọi câu hỏi tương đương đều dùng cùng một công thức, giảm sai khác giữa các lần sinh SQL.

2.5. Agentic Text-to-SQL

Agentic text-to-SQL mở rộng RAG bằng vòng lặp ra quyết định và sử dụng công cụ. Agent có thể truy hỏi metadata, hỏi lại người dùng, sinh SQL, kiểm tra, thực thi, nhận lỗi, sửa truy vấn, tạo biểu đồ và tóm tắt. Vanna 2.0 hiện thực hóa mô hình này qua **Agent**, **ToolRegistry**, **RunSqlTool**, bộ phân giải người dùng và các thành phần UI streaming [2].

3. Những thách thức cốt lõi của Text-to-SQL

3.1. Thiếu ngữ cảnh đặc thù doanh nghiệp

LLM không thể suy ra rằng một mã nội bộ, một view tổng hợp hoặc một điều kiện lọc cụ thể đại diện cho khái niệm nghiệp vụ nào. Nếu chỉ cung cấp tên bảng và cột, truy vấn có thể chạy nhưng trả lời sai ý nghĩa. Nghiên cứu trong repo Vanna báo cáo độ chính xác trong thí nghiệm cũ tăng từ khoảng 3% khi chỉ dùng schema lên khoảng 80% khi truy hỏi ngữ cảnh liên quan gồm schema, tài liệu và ví dụ SQL; đây là kết quả lịch sử trên bộ thử nghiệm cụ thể, không nên hiểu là cam kết hiệu năng cho mọi hệ thống [4].

3.2. Ý định người dùng mơ hồ

Câu hỏi “sản phẩm bán chạy nhất” có thể có nhiều cách hiểu: số lượng bán, số đơn hàng, doanh thu hay lợi nhuận; có tính đơn hoàn hay không; lấy một hay nhiều sản phẩm. Một hệ thống đáng tin cậy phải:

- phát hiện thuật ngữ chưa được định nghĩa;
- đối chiếu với semantic layer và hồ sơ người dùng;
- hỏi một câu làm rõ ngắn gọn khi có nhiều diễn giải hợp lý;
- lưu lựa chọn đã xác nhận làm ngữ cảnh cho các câu hỏi sau.

3.3. Giới hạn sinh mã và khác biệt phương ngữ

Các hệ quản trị dùng hàm và cú pháp khác nhau. Ví dụ, cách lấy tháng từ timestamp trong BigQuery khác MySQL. Ngoài lỗi cú pháp, LLM còn có thể bịa tên bảng/cột, join sai khóa, nhân bản bản ghi khi tổng hợp, hoặc bỏ điều kiện phân quyền. Do đó, prompt cần nêu rõ dialect, còn pipeline phải có cơ chế kiểm chứng không phụ thuộc LLM [1].

3.4. Rủi ro an toàn và quản trị

Một truy vấn đúng cú pháp vẫn có thể gây rủi ro nếu đọc dữ liệu ngoài phạm vi người dùng, quét toàn bảng, làm lộ thông tin cá nhân hoặc thực hiện DDL/DML. Các kiểm soát tối thiểu gồm:

- tài khoản cơ sở dữ liệu chỉ đọc và nguyên tắc quyền tối thiểu;
- allowlist schema/view, chặn nhiều câu lệnh và từ khóa ghi/xóa;
- row-level security hoặc bộ lọc theo nhóm người dùng;
- timeout, giới hạn số hàng, chi phí quét và tốc độ yêu cầu;
- audit log cho câu hỏi, SQL, người dùng, thời gian và kết quả kiểm tra.

4. Phương pháp luận nâng cao độ chính xác

4.1. Giai đoạn 1: Chuẩn hóa và lập chỉ mục ngữ cảnh

1. **Thu thập metadata:** DDL, khóa, view, comment, data dictionary, chủ sở hữu dữ liệu và dialect.
2. **Chuẩn hóa ngữ nghĩa:** KPI, thuật ngữ viết tắt, điều kiện lọc, quy tắc thời gian và đơn vị đo.
3. **Xây kho ví dụ chuẩn:** câu hỏi-SQL do chuyên gia dữ liệu xác nhận; loại bỏ truy vấn lỗi thời hoặc có quyền truy cập không phù hợp.
4. **Tạo chỉ mục truy hồi:** embedding cho câu hỏi, mô tả bảng/cột và tài liệu; lưu metadata để lọc theo miền dữ liệu và quyền.
5. **Quản lý phiên bản:** liên kết mỗi mục ngữ cảnh với phiên bản schema và ngày xác nhận.

4.2. Giai đoạn 2: Truy hồi và xếp hạng đa tầng

Không nên đưa toàn bộ data warehouse vào prompt. Quy trình đề xuất:

1. xác định miền dữ liệu bằng phân loại ý định;
2. truy hồi dataset/table/cột theo tương đồng ngữ nghĩa;
3. bổ sung quan hệ khóa và các bảng cầu nối cần thiết;
4. truy hồi ví dụ câu hỏi-SQL tương tự và quy tắc nghiệp vụ;
5. xếp hạng lại dựa trên độ liên quan, độ mới, quyền và mức tin cậy;
6. đóng gói tập ngữ cảnh nhỏ, có cấu trúc vào prompt.

Nguyên tắc “đủ nhưng không thừa”

Ngữ cảnh quá ít làm mô hình bịa schema; ngữ cảnh quá nhiều làm tăng độ trễ, chi phí và nhiễu. Mục tiêu là truy hồi tập bảng/cột/ví dụ nhỏ nhất vẫn đủ để giải quyết câu hỏi.

4.3. Giai đoạn 3: Làm rõ ý định và phân giải thực thể

Trước khi sinh SQL, agent đánh giá khả năng trả lời:

- Nếu câu hỏi rõ và schema đủ: tiếp tục sinh SQL.
- Nếu thiếu định nghĩa: hỏi lựa chọn cụ thể, ví dụ “bán chạy theo số lượng hay doanh thu?”.
- Nếu thực thể không khớp: tìm giá trị gần đúng hoặc đề nghị người dùng xác nhận.
- Nếu vượt quyền hoặc ngoài miền dữ liệu: từ chối có giải thích.

4.4. Giai đoạn 4: Sinh SQL có ràng buộc

Prompt sinh SQL nên chứa: vai trò hệ thống; dialect; schema được truy hồi; định nghĩa nghiệp vụ; ví dụ tương tự; chính sách an toàn; yêu cầu định dạng đầu ra. Có thể yêu cầu mô hình tạo một cấu trúc gồm SQL, bảng/cột sử dụng và giả định, nhưng chỉ SQL đã được kiểm chứng mới được thực thi.

4.5. Giai đoạn 5: Kiểm chứng, dry-run và tự sửa

Bảng 2: Chuỗi kiểm chứng đề xuất trước khi trả kết quả

Lớp kiểm tra	Nội dung	Tín hiệu thất bại
Chính sách	Chỉ SELECT, schema cho phép, quyền theo người dùng	Vi phạm quyền/từ khóa cấm
Cú pháp	Parse AST, một câu lệnh, đúng dialect	Parser báo lỗi
Ngữ nghĩa tĩnh	Bảng/cột tồn tại, join có khóa, KPI đúng định nghĩa	Định danh không tồn tại
Chi phí	EXPLAIN/dry-run, giới hạn scan, timeout và số hàng	Chi phí vượt ngưỡng
Thực thi giới hạn	Chạy read-only với LIMIT và timeout	Lỗi DB hoặc kết quả bất thường
Tự sửa	Đưa lỗi có cấu trúc lại cho LLM, giới hạn số vòng	Vượt số lần sửa

Google Cloud nhận định parser và dry-run cung cấp tín hiệu xác định để bổ sung cho quy trình dựa trên mô hình; lỗi này có thể được đưa lại vào prompt để mô hình sửa ở vòng tiếp theo [1].

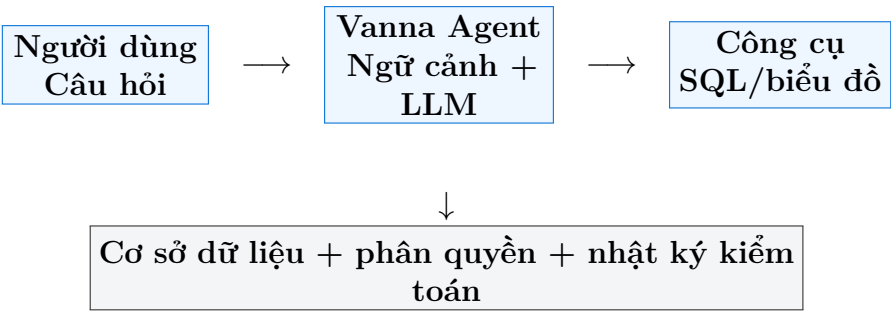
4.6. Self-consistency và lựa chọn ứng viên

Với câu hỏi quan trọng, hệ thống có thể sinh nhiều ứng viên bằng prompt hoặc mô hình khác nhau. Mỗi ứng viên được chấm theo: hợp lệ cú pháp, tương thích schema, tuân thủ chính sách, kết quả dry-run, độ tương đồng giữa kết quả và mức đồng thuận. Cách này tăng độ tin cậy nhưng làm tăng độ trễ và chi phí, do đó chỉ nên bật theo mức rủi ro.

5. Kiến trúc Vanna và ánh xạ với phương pháp Google Cloud

5.1. Các thành phần chính của Vanna 2.0

Theo README của repo, Vanna 2.0 cung cấp giao diện <vanna-chat>, FastAPI/Flask backend, agent, tool, bảng và biểu đồ streaming, nhận diện người dùng, row-level security, audit log và lifecycle hook. Hệ thống hỗ trợ nhiều LLM và nhiều loại cơ sở dữ liệu [2].



5.2. Vanna 1.x và Vanna 2.0

Bảng 3: Phân biệt hai cách tiếp cận trong hệ sinh thái Vanna

Tiêu chí	Vanna 1.x / Legacy	Vanna 2.0
Mô hình chính	RAG cho sinh SQL	Agent dùng công cụ và UI streaming
Ngữ cảnh	DDL, tài liệu, câu hỏi-SQL trong vector store	System prompt, memory, tool schema, middleware
Luồng xử lý	Train metadata, ask, sinh SQL, chạy và hiển thị	Nhận diện user, agent loop, tool call, component streaming
Bảo mật	Chủ yếu do ứng dụng tích hợp	User-aware tools, group permission, hooks và audit
Phù hợp	PoC RAG, notebook, tích hợp kiểu cũ	Ứng dụng hội thoại dữ liệu production

Repo Vanna được đánh dấu lưu trữ (read-only) trên GitHub vào ngày 29/03/2026 tại thời điểm truy cập báo cáo. Vì vậy, khi triển khai cần khóa phiên bản, kiểm toán dependency và lập kế hoạch bảo trì nội bộ hoặc đánh giá nhánh kế nhiệm [2].

5.3. Bảng ánh xạ kỹ thuật

Bảng 4: Ánh xạ kỹ thuật Google Cloud sang thành phần triển khai Vanna

Kỹ thuật	Mục đích	Cách hiện thực đề xuất
Truy hỏi schema	Chọn bảng/cột liên quan	Vector store/metadata catalog, filter theo miền và quyền
In-context examples	Hướng dẫn join và KPI đúng	Memory chứa cặp câu hỏi-SQL đã xác nhận
Lớp ngữ nghĩa	Đồng nhất ngôn ngữ nghiệp vụ	View phân tích, metric catalog, tài liệu trong prompt
Phân giải thực thể	Ánh xạ tên gọi sang giá trị dữ liệu	Tool tìm kiếm giá trị, fuzzy matching, bước xác nhận
Disambiguation	Không tự đoán câu hỏi mơ hồ	Agent sinh câu hỏi làm rõ trước khi gọi SQL tool
Kiểm chứng/sửa lỗi	Phát hiện và sửa lỗi	Middleware hoặc tool parser/dry-run, trả lỗi có cấu trúc cho agent
Self-consistency	Giảm phụ thuộc một lần sinh	Sinh k ứng viên, kiểm tra và chọn theo điểm tin cậy
User-aware security	Bảo vệ dữ liệu theo người dùng	UserResolver, nhóm quyền, RLS và SQL policy tool

6. Phương án triển khai hệ thống đề xuất

6.1. Kiến trúc logic

1. **Kênh người dùng:** web chat hoặc tích hợp cổng nghiệp vụ.
2. **API và xác thực:** FastAPI nhận cookie/JWT, ánh xạ thành **User** và nhóm quyền.
3. **Orchestrator:** Vanna Agent quản lý hội thoại, truy hỏi ngữ cảnh, gọi LLM và công cụ.
4. **Context service:** metadata catalog, semantic layer, vector index và kho ví dụ SQL đã xác nhận.
5. **Validation gateway:** parser, chính sách SQL, dry-run, timeout, LIMIT và vòng sửa lỗi.
6. **Data access:** tài khoản chỉ đọc, view nghiệp vụ, RLS và cache.
7. **Quan sát hệ thống:** audit log, trace, latency, token, lỗi và phản hồi đúng/sai.

6.2. Luồng xử lý trực tuyến

Bảng 5: Quy trình xử lý một câu hỏi

Bước	Thành phần	Xử lý
1	Web/API	Nhận câu hỏi, danh tính, lịch sử hội thoại
2	Intent router	Xác định miền dữ liệu và mức mơ hồ
3	Context retriever	Lấy bảng/cột, tài liệu, ví dụ và quyền liên quan
4	Agent/LLM	Hỏi làm rõ hoặc sinh ứng viên SQL theo dialect
5	Validation gateway	Kiểm tra quyền, parse, dry-run và chi phí
6	SQL tool	Thực thi read-only trong giới hạn tài nguyên
7	Agent	Kiểm tra kết quả, sửa lỗi nếu cần, tạo diễn giải
8	UI	Stream bảng, biểu đồ, SQL (nếu được phép) và tóm tắt
9	Audit/eval	Lưu trace và phản hồi để cải thiện kho ví dụ

6.3. Khảo nghiệm hiện tại

PoC đã được triển khai và kiểm thử theo cấu hình: **FastAPI + Vanna 2.0 + Ollama + PostgreSQL trên Neon**. Ollama chạy llama3.2 tại máy cục bộ nên không phát sinh chi phí API LLM. Thành phần Vanna sử dụng **PostgresRunner** để kết nối tới chuỗi `DATABASE_URL` của Neon có bật SSL. Với ứng dụng, chuỗi kết nối pooled được ưu tiên nhằm quản lý số lượng kết nối hiệu quả [5].

Bảng 6: Cấu trúc dữ liệu PostgreSQL trong khảo nghiệm

Nhóm	Bảng và chức năng	Số bản ghi
Khách hàng	<code>regions, customers</code>	5; 500
Danh mục hàng hóa	<code>categories, suppliers, products</code>	4; 8; 30
Giao dịch	<code>sales_channels, orders, order_items</code>	3; 10.000; 20.000
Thanh toán và hoàn hàng	<code>payments, returns</code>	10.000; 801

Mười bảng được liên kết bằng khóa chính và khóa ngoại. View `sales_enriched` hợp nhất khách hàng, khu vực, sản phẩm, danh mục, nhà cung cấp, kênh bán, thanh toán và hoàn hàng thành 20.000 dòng phân tích. Dữ liệu trải từ tháng 01/2025 đến tháng 06/2026. Các chỉ tiêu doanh thu và lợi nhuận chỉ tính đơn có `status = 'Completed'`; tỷ lệ hoàn hàng dùng tập đơn hoặc số lượng trả tùy định nghĩa chỉ tiêu.

Do model local có thể phát tool call dưới dạng chuỗi JSON thay vì cấu trúc function calling, lớp tích hợp được bổ sung bộ phân tích tool call dạng text. Các truy vấn nghiệp vụ trọng yếu có mẫu SQL xác định; yêu cầu đồng thời trong một hội thoại bị khóa ở phía API; file CSV của truy vấn gần nhất được theo dõi để công cụ biểu đồ luôn đọc đúng kết

quả. Đây là các biện pháp ổn định hóa phù hợp với model nhỏ, không thay thế validation gateway ở môi trường production.

Listing 1: Ví dụ view ngữ nghĩa giúp model local sinh SQL ổn định

```
SELECT
  order_date,
  region,
  product_name,
  quantity,
  revenue,
  profit
FROM sales_enriched
WHERE status = 'Completed';
```

6.4. Chiến lược bảo mật

1. Trước production, tạo role **vanna_reader** chỉ có quyền **CONNECT**, **USAGE** schema và **SELECT**; không dùng role quản trị của Neon cho agent.
2. Chỉ cho phép truy vấn các view đã kiểm duyệt; che cột nhạy cảm.
3. Áp chính sách RLS tại cơ sở dữ liệu, không chỉ trong prompt.
4. SQL phải qua parser và policy engine trước khi thực thi.
5. Không gửi dữ liệu mẫu nhạy cảm ra LLM bên ngoài nếu chưa có phê duyệt.
6. Mã hóa kết nối, quản lý secret tập trung và xoay vòng khóa.
7. Lưu audit bất biến và cơ chế truy vết từ câu trả lời về SQL/nguồn.

7. Phương pháp đánh giá và tiêu chí nghiệm thu

7.1. Bộ dữ liệu kiểm thử

Xây bộ câu hỏi do chuyên gia nghiệp vụ xác nhận, phân tầng theo độ khó:

- truy vấn một bảng, filter và tổng hợp đơn giản;
- join nhiều bảng, thời gian, top-k và so sánh kỳ;
- thuật ngữ nghiệp vụ, entity gần đúng và câu hỏi mơ hồ;
- truy vấn có nguy cơ vượt quyền hoặc tiêu tốn tài nguyên;
- câu hỏi không thể trả lời từ dữ liệu hiện có.

Mỗi mẫu cần có SQL chuẩn hoặc tập kết quả chuẩn, các cách diễn đạt tương đương, quyền người dùng, dialect và mức độ chấp nhận.

7.2. Hệ chỉ số

Bảng 7: Các chỉ số đánh giá đề xuất

Chỉ số	Ý nghĩa	Cách đo
Execution success	SQL chạy được	Tỷ lệ không lỗi DB
Result accuracy	Kết quả đúng với đáp án	So sánh tập kết quả
Semantic correctness	Đúng KPI, filter và phạm vi	Chuyên gia/LLM-judge có rubric
Clarification quality	Hỏi lại đúng khi mơ hồ	Precision/recall phát hiện mơ hồ
Safety rate	pass Không vượt quyền/chính sách	Bộ test tấn công và policy logs
Latency	Thời gian phản hồi	P50/P95 theo độ khó
Repair rate	Tỷ lệ sửa thành công sau lỗi	Số vòng và tỷ lệ hoàn tất
User acceptance	Người dùng chấp nhận câu trả lời	Phản hồi đúng/sai, CSAT

Exact string match chỉ phù hợp để tham khảo vì hai câu SQL khác nhau có thể trả cùng kết quả. Do đó, trọng tâm nên là execution accuracy, result accuracy và semantic correctness. Google Cloud khuyến nghị kết hợp metric người dùng, offline eval, đánh giá con người và LLM-as-a-judge; LLM-as-a-judge phải dùng rubric rõ và không thay thế hoàn toàn chuyên gia [1].

7.3. Thiết kế thử nghiệm đối chứng

Để đo đóng góp của từng kỹ thuật, thực hiện ablation test:

1. Baseline chỉ có schema.
2. Thêm ví dụ SQL tĩnh.
3. Thêm truy hồi ví dụ theo ngữ nghĩa.
4. Thêm semantic layer và entity resolution.
5. Thêm disambiguation.
6. Thêm validation/repair.
7. Thêm self-consistency cho nhóm câu hỏi rủi ro cao.

Mỗi cấu hình chạy cùng bộ câu hỏi, cùng model và tham số. Báo cáo chênh lệch độ chính xác, độ trễ, chi phí và tỷ lệ vi phạm chính sách.

8. Kế hoạch thực hiện và quản trị rủi ro

8.1. Lộ trình bốn giai đoạn

Bảng 8: Lộ trình triển khai đề xuất

Giai đoạn	Mục tiêu	Công việc chính	Đầu ra
1. Chuẩn bị	Chọn miền dữ liệu hẹp	Kiểm kê schema, KPI, quyền; xây 50-100 câu hỏi chuẩn	Data contract và bộ eval v1
2. PoC	Chứng minh luồng end-to-end	Vanna agent, 1 DB read-only, RAG cơ bản, validation	Demo nội bộ và báo cáo baseline
3. Pilot	Kiểm thử với người dùng thật	RLS, audit, feedback, monitoring, ablation test	Chỉ số P50/P95 và acceptance
4. Production	Vận hành có kiểm soát	HA, cache, secret, CI/CD eval, rollback, SOP	SLA, dashboard và quy trình hỗ trợ

8.2. Ma trận rủi ro

Bảng 9: Rủi ro chính và biện pháp giảm thiểu

Rủi ro	Mức độ	Biện pháp
Bị bỏ/bảng/cột SQL nhưng sai KPI	Cao	Truy hồi schema, parser, kiểm tra catalog, repair
Rò rỉ dữ liệu	Cao	Semantic layer, golden query, review nghiệp vụ
Truy vấn tốn tài nguyên	Rất cao	RLS tại DB, view kiểm duyệt, masking, audit
Schema thay đổi	Cao	Dry-run/EXPLAIN, timeout, LIMIT, quota
Model yếu	Trung bình	Version metadata, phát hiện drift, CI eval
Repo ngừng bảo trì	Trung bình	Context ngắn và rõ, view ngữ nghĩa, model routing
	Trung bình	Pin phiên bản, fork nội bộ, SBOM và kế hoạch thay thế

9. Kết luận và đề xuất hành động

Text-to-SQL không phải bài toán “chọn một LLM rồi yêu cầu viết SQL”. Hệ thống đáng tin cậy phải kết hợp ngữ cảnh doanh nghiệp, truy hồi thông minh, làm rõ ý định, sinh SQL theo dialect, kiểm chứng xác định, phân quyền và đánh giá liên tục. Google Cloud cung cấp

khung kỹ thuật nâng độ chính xác; Vanna cung cấp nền tảng triển khai agent, công cụ SQL, UI và tích hợp hệ thống.

Kết quả hiện thực cho thấy kiến trúc có thể vận hành với LLM local và cơ sở dữ liệu PostgreSQL từ xa: giao diện nhận câu hỏi tiếng Việt, agent gọi công cụ SQL, Neon trả dữ liệu, sau đó hệ thống hiển thị bảng hoặc biểu đồ. Việc chuyển từ SQLite sang 10 bảng PostgreSQL cũng cho phép kiểm thử join, tổng hợp, lợi nhuận và hoàn hàng trong mô hình gần với hệ thống nghiệp vụ hơn.

Đề xuất triển khai:

1. Nghiệm thu PoC Neon PostgreSQL hiện tại trên bộ dữ liệu tổng hợp, sau đó tạo role chỉ đọc trước khi kết nối dữ liệu nghiệp vụ.
2. Xây semantic layer và bộ 100 câu hỏi-SQL chuẩn trước khi mở rộng model.
3. Bổ sung validation gateway và RLS trước khi cho người dùng nghiệp vụ truy cập.
4. Đo result accuracy, safety và P95 latency theo từng phiên bản; chỉ phát hành nếu không hồi quy trên bộ eval.
5. Do repo Vanna đã ở trạng thái archive, thực hiện đánh giá license, dependency và phương án fork/bảo trì nội bộ trước production.

Kết luận cuối cùng

Giá trị cạnh tranh nằm ở chất lượng lớp ngữ nghĩa, kho ví dụ đã xác nhận và cơ chế kiểm chứng, không nằm riêng ở model. Phương án phù hợp là dùng Vanna làm lớp điều phối có thể thay thế, giữ metadata, policy và bộ eval ở các thành phần độc lập để giảm khóa chặt công nghệ.

Người thực hiện
(Đã lập báo cáo)

Lê Đức Anh

Tài liệu tham khảo

- [1] Per Jacobsson, *Getting AI to write good SQL: Text-to-SQL techniques explained*, Google Cloud Blog, 16/05/2025. <https://cloud.google.com/blog/products/databases/techniques-for-improving-text-to-sql>. Truy cập ngày 30/07/2026.
- [2] Vanna AI, *Vanna 2.0: Turn Questions into Data Insights*, GitHub repository `vanna-ai/vanna`, phiên bản mã nguồn tham chiếu 2.0.2 trong môi trường khảo nghiệm. <https://github.com/vanna-ai/vanna>. Truy cập ngày 30/07/2026.
- [3] Vanna AI, *Vanna Legacy README: RAG framework for SQL generation*, tệp `README_LEGACY.md` trong repository Vanna. https://github.com/vanna-ai/vanna/blob/main/README_LEGACY.md. Truy cập ngày 30/07/2026.
- [4] Vanna AI, *AI SQL Accuracy: Testing different LLMs and context strategies to maximize SQL generation accuracy*, 17/08/2023. <https://github.com/vanna-ai/vanna/blob/main/papers/ai-sql-accuracy-2023-08-17.md>. Truy cập ngày 30/07/2026.
- [5] Neon, *Connection pooling*, tài liệu kết nối PostgreSQL và pooled connection cho ứng dụng. <https://neon.com/docs/connect/connection-pooling>. Truy cập ngày 31/07/2026.